14 haftalık yol haritasında 11. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 11

Diziler II: Çok Boyutlu Diziler ve Matris Düşüncesi

İki boyutlu diziler ve satır öncelikli yerleşim; fonksiyona aktarım; matris işlemleri; ızgara, komşuluk ve sınır denetimi; görüntü filtreleri, Hayat Oyunu ve önbellek deneyi

MODÜL V · VERİ TOPLULUKLARI VE BELLEK · H10–H12

> **HAFTANIN BÜYÜK SORUSU**
>
> *Bir fotoğraf, bir satranç tahtası ya da bir sınıf listesi bilgisayar için nedir ve aynı matrisi satır satır ya da sütun sütun dolaşmak neden farklı hızda sonuç verir?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 2
2. Geçen Haftadan: Şeritten Izgaraya 3
3. Kavramsal Anlatım 4
3.1 Neden iki boyut? Tablolar ve ızgaralar 4
3.2 Bildirim, ilk değer ve sizeof 4
3.3 Erişim ve iç içe döngü: satır satır, sütun sütun 4
3.4 Kaputun altında: iki boyutlu dizi bellekte tek şerittir 5
3.5 İki boyutlu diziyi fonksiyona aktarmak 6
3.6 Komşuluk ve sınır denetimi 7
3.7 Görüntü = piksel matrisi 7
3.8 Kaputun altında: önbellek ve dolaşma sırası 8
4. Çözümlü Örnekler: Örnek Merdiveni 10
5. Hata Avcısı 21
6. Mühendis Gözüyle 23
7. Mini-OBS Adımı: v0.11 — Öğrenci × Ders Not Matrisi 24
8. Alıştırmalar ve Atölye 25
9. Kendini Değerlendirme 28
10. AP-2 Köprüsü ve Merak Kutusu 29
11. Özet, Kavram Haritası ve Terimler 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir fotoğraf, bir satranç tahtası ya da bir sınıf listesi bilgisayar için nedir ve aynı matrisi satır satır ya da sütun sütun dolaşmak neden farklı hızda sonuç verir?

H10'da yüz notu **tek bir dizide** tuttuk: bitişik kutulardan oluşan bir şerit. Ama verinin çoğu **tablodur**: sınıf listesi (öğrenci × ders), sinema salonu (sıra × koltuk), satranç tahtası, fotoğraf (satır × sütun piksel). Bu hafta diziyi ikinci boyuta taşıyoruz: **iki boyutlu dizi** (two-dimensional array), matematikçilerin adıyla **matris**. En önemli gerçek şaşırtıcı derecede sadedir: bellek hâlâ tek bir şerittir; ızgara yalnızca **kafamızdaki** modeldir. Aynı matrisi satır satır ve sütun sütun dolaşmanın neden **farklı hızda** sonuç verdiğini de bu gerçek açıklayacak.

GERÇEK DÜNYADAN · **1970: Hayat Oyunu ve piksel matrisleri**

Ekim 1970'te **Martin Gardner**, Scientific American dergisindeki "Mathematical Games" köşesinde **John Horton Conway**'in tasarladığı bir "tek kişilik oyunu" tanıttı: **Hayat Oyunu** (Game of Life) (Gardner, 1970). Kareli düzlemde her hücre **canlı** ya da **ölüdür**; sonraki nesil sekiz komşuya bakan üç kuralla belirlenir: 2 ya da 3 canlı komşusu olan canlı hücre **yaşar**; 4 ve daha çok komşusu olan kalabalıktan, 0 ya da 1 komşusu olan yalnızlıktan **ölür**; tam 3 canlı komşusu olan boş hücrede **doğum** olur (kısaca B3/S23). Beş hücrelik **planör** (glider) dört nesilde bir kendini yeniden kurarak bir hücre çapraz kayar. Program olarak Hayat Oyunu, **iki boyutlu bir dizi** ve **iç içe döngülerden** ibarettir (Örnek 13).

Her **gri tonlu fotoğraf** da bir sayı matrisidir: piksel başına genellikle 1 bayt, 0 (siyah) … 255 (beyaz). 1920 × 1080 bir kare 2 073 600 piksellik bir matristir; renkli görüntüde her pikselin üç sayısı (kırmızı, yeşil, mavi) vardır. "Negatif", "ayna", "bulanıklaştır" düğmeleri bu matris üzerinde iç içe döngülerdir (Örnek 11).

Öğrenme çıktıları. Bu haftanın sonunda:

ÖÇ 11.1 İki boyutlu dizileri bildirir, satır–sütun indisleriyle erişir ve iç içe döngülerle işler. *(Bölüm 3.1–3.3 ve 4.1–4.3)*

ÖÇ 11.2 Çok boyutlu dizilerin satır öncelikli (row-major) yerleşimini açıklar ve eleman adresini hesaplar. *(Bölüm 3.4, 3.8 ve 4.12)*

ÖÇ 11.3 Matris işlemlerini (toplama, devrik, çarpma, köşegen, satır/sütun toplamları) gerçekler. *(Bölüm 4.3–4.6 ve 4.8)*

ÖÇ 11.4 Izgara tabanlı problemleri (oyun tahtası, koltuk planı, görüntü) iki boyutlu dizilerle modeller; komşuluk ve sınır denetimlerini doğru uygular. *(Bölüm 3.6, 3.7, 4.7, 4.9–4.11, 4.13 ve 4.14)*

2. Geçen Haftadan: Şeritten Izgaraya

H10'un dört taşı bu haftanın temelidir: (1) `int a[5]` **bitişik** beş kutudur, `&a[i]` = taban + i × `sizeof(int)`; (2) dizi fonksiyona **adresiyle** gider, boyut ayrı parametredir; (3) **frekans dizisinde** değer indis olur (`frekans[zar]++`); (4) sınırı aşmak **tanımsız davranıştır** (Heartbleed; Durumeric vd., 2014). Üç geri çağırma sorusu (`kodlar/quiz/`):

TAHMİN ET · **Geri çağırma: üç soru**

- **S1 (indis–adres).** `H11_Q01_indis_adres.c`: `int a[5]` için `&a[3]`, `&a[0]`'dan kaç bayt uzaktadır? (Programdaki `(char *)` dönüşümü ve `%td` ile yazdırılan adres farkı yalnızca bayt uzaklığını yazdırmanın yoludur; ayrıntısı H12'de.)
- **S2 (diziyi fonksiyona aktarma).** `H11_Q02_sizeof_tuzagi.c`'de `main` `sizeof notlar / sizeof notlar[0]` ile 6 bulur. `dizi_max`'in içinde aynı ifade yazılsaydı (64 bit Linux) kaç çıkardı?
- **S3 (frekans dizisi).** `H11_Q03_frekans.c` 8 zar atışını sayar ve `frekans[3]`, `frekans[6]`, `frekans[4]`'ü yazdırır. Çıktı nedir?

Yanıtlar (gerçek). S1: **12** (3 × 4 bayt). S2: **2**: parametre `const int a[]` aslında bir **adrestir** (8 / 4 = 2); GCC de uyarır: "'sizeof' on array function parameter 'a' will return size of 'const int *' [-Wsizeof-array-argument]". S3: "**3 2 0**". Bu hafta aynı soruları iki boyutta soracağız; frekans dizisi de **frekans matrisine** dönüşecek (Mini-OBS'nin ders × harf tablosu).

3. Kavramsal Anlatım

3.1 Neden iki boyut? Tablolar ve ızgaralar

Beş öğrencinin dört dersteki notlarını H10 ile tutmak için dört ayrı dizi ya da yirmi elemanlı tek bir dizi ve "3. öğrencinin 2. dersi, indisi 2 × 4 + 1 = 9 olan elemandır" gibi elle hesap gerekirdi. **İki boyutlu dizi** bu hesabı derleyiciye bırakır: `int notlar[5][4];` "5 satır, her satırda 4 sütun" demektir ve `notlar[2][1]` doğrudan 3. öğrencinin 2. dersidir. Analoji: **sinema salonu**. Bilet "sıra 3, koltuk 4" der; ilk iki sırayı tamamen geçip üçüncü sırada dördüncü koltuğa gelirsiniz. Satır = sıra, sütun = koltuk; C'de ikisi de **0'dan** başlar. Bu haftanın modellerinde (not tablosu, sinema salonu, tic-tac-toe tahtası, gri tonlu görüntü, Hayat Oyunu) ilk indis hep satır, ikinci indis sütundur.

3.2 Bildirim, ilk değer ve sizeof

Genel biçim `tip ad[SATIR][SUTUN];`; boyutlar derleme zamanı sabitleridir (`#define SATIR 3`). İlk değer listesi **satır satır** yazılır; eksik elemanlar 0 olur, `{0}` bütün matrisi sıfırlar; liste verilirse **yalnızca ilk boyut** boş bırakılabilir (`H11_K01_ilk_deger.c`):

H11_K01_ilk_deger.c · satır 26–29

```
26      int a[2][3] = {{1, 2, 3}, {4, 5, 6}};     /* satır satır */
27      int c[2][3] = {{1}, {4, 5}};              /* kısmi: kalanlar 0 */
28      int d[2][3] = {0};                        /* hepsi 0 */
29      int e[][3] = {{1, 2, 3}, {4, 5, 6}, {7}}; /* satır sayısı: 3 */
```

ÇIKTI (GERÇEK)

```
$ ./ilk_deger
a:  { 1 2 3 }  { 4 5 6 }
c:  { 1 0 0 }  { 4 5 0 }
d:  { 0 0 0 }  { 0 0 0 }
e:  { 1 2 3 }  { 4 5 6 }  { 7 0 0 }
sizeof a = 24, sizeof a[0] = 12, sizeof a[0][0] = 4
satır sayısı = 3, sütun sayısı = 3
```

`sizeof a` bütün matrisin (24 bayt), `sizeof a[0]` **bir satırın** (12), `sizeof a[0][0]` bir elemanın (4) boyutudur; satır sayısı `sizeof e / sizeof e[0]`, sütun sayısı `sizeof e[0] / sizeof e[0][0]` (yalnızca dizinin bildirildiği kapsamda, H10). İç süslü parantezler atlanabilir ama GCC `-Wall` uyarır ve kısmi listelerde değerler **yanlış satıra** kayar (Hata Avcısı 6).

3.3 Erişim ve iç içe döngü: satır satır, sütun sütun

`m[i][j]` sıradan bir değişken gibi okunur, atanır, `scanf("%d", &m[i][j])` ile doldurulur. Bütün matris H7'nin **iç içe döngüsüyle** işlenir: dış döngü satırları (`i`), iç döngü sütunları (`j`) dolaşır; gövde SATIR × SUTUN kez çalışır. Döngüler yer değiştirirse aynı elemanlar **sütun sütun** dolaşılır. Toplam gibi sıradan bağımsız sonuçlar değişmez; ama biri bellekte ardışık, öteki **atlayarak** ilerler (`H11_K05_dolasma_sirasi.c` iki sıradaki bayt uzaklıklarını yazdırır):

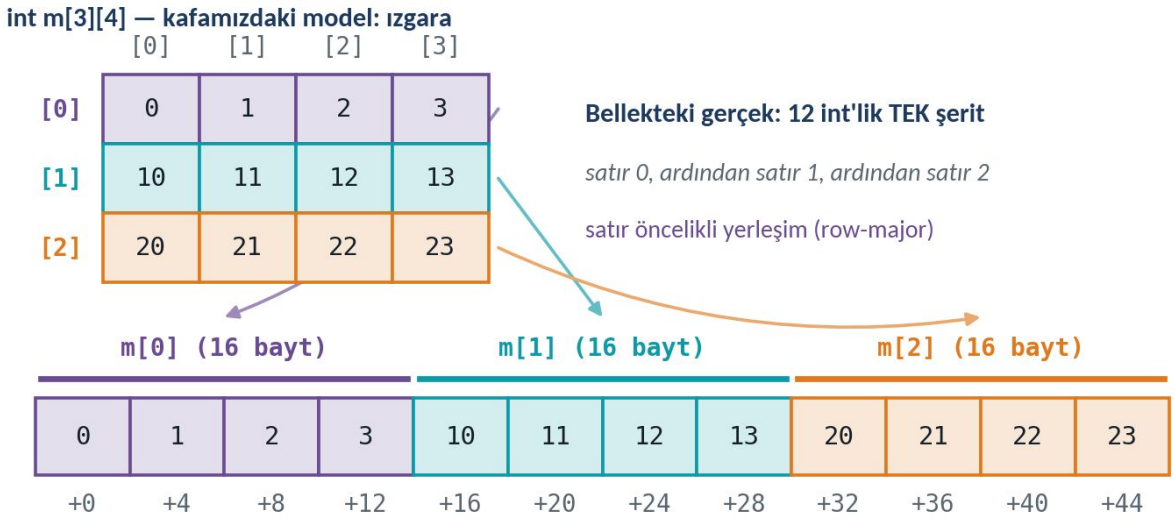
ÇIKTI (GERÇEK)

```
$ ./dolasma_sirasi
satır öncelikli: 0 4 8 12 16 20 24 28 32 36 40 44
sütun öncelikli: 0 16 32 4 20 36 8 24 40 12 28 44
```

3.4 Kaputun altında: iki boyutlu dizi bellekte tek şerittir

Bellek tek boyutlu bir bayt dizisidir; "ızgara" diye bir donanım yoktur. C çok boyutlu diziyi **dizilerin dizisi** olarak tanımlar: `int m[3][4]`, her biri 4 `int`'lik **3 elemanlı bir dizidir** ve elemanlar art arda durur (ISO/IEC, 2018, 6.5.2.1; Kernighan & Ritchie, 1988, Bölüm 5.7; King, 2008, Bölüm 8). Sonuç **satır öncelikli yerleşimdir** (row-major order): 0. satırın 4 elemanı, hemen ardından 1. satırınkiler, sonra 2. satırınkiler (Şekil 11.1). `H11_K02_bellek_serit.c` 12 elemanın gerçek adresini (`%p`) ve `&m[0][0]`'dan uzaklığını (`(char *)&m[i][j] - taban`; `(char *)` dönüşümü ve işaretçi farkı burada yalnızca bayt uzaklığını ölçmenin yoludur, ayrıntısı H12'de) yazdırır:

Şekil 11.1
int m[3][4]: kafamızdaki ızgara ve bellekteki tek şerit; alttaki sayılar &m[0][0]'dan bayt uzaklığı (H11_K02, gerçek çalıştırma)



&m[0][0] = 0x7ffd9dd7e190 (gerçek çalıştırma) · alttaki sayılar: &m[0][0]'dan bayt uzaklığı

ÇIKTI (GERÇEK, KESİT)

```
$ ./bellek_serit      # 1. çalıştırma (ilk 4 ve son 4 satır)
eleman   adres            uzaklık  (i*SUTUN+j)*4
m[0][0]  0x7ffd9dd7e190     0         0
m[0][1]  0x7ffd9dd7e194     4         4
m[0][2]  0x7ffd9dd7e198     8         8
…
&m[1][0] - &m[0][3] = 4 bayt (1 eleman)
&m[1][0] - &m[0][2] = 8 bayt (2 eleman)
sizeof m = 48, sizeof m[1] = 16 (bir satır)
m[1] = 0x7ffd9dd7e1a0 = &m[1][0] (satırın adresi)
```

Gözlemler: (1) her eleman bir öncekinden **4 bayt** ötededir, satır sınırında da: `&m[1][0] − &m[0][3]` = **4 bayt**; `&m[1][0] − &m[0][2]` = **8 bayt** (arada yalnızca `m[0][3]`). (2) Ölçülen uzaklık her elemanda formülle aynıdır (son iki sütun). (3) `sizeof m[1]` = 16: **bir satır** 16 baytlık bir bloktur ve `m[1]` bu satırın başlangıç adresidir (H12'de döneceğiz). (4) İkinci çalıştırmada `&m[0][0]` 0x7ffd9dd7e190 yerine **0x7ffe123f5950** çıktı (ASLR, H9); uzaklıklar değişmedi.

KAPUTUN ALTINDA · **Adres formülü**

adres(m[i][j]) = taban + (i × SUTUN + j) × sizeof(eleman): önce **i tam satır** (i × SUTUN eleman) atlanır, sonra satır içinde **j** eleman ilerlenir. Formülde **SATIR yoktur ama SUTUN vardır**; bu, fonksiyonlara aktarımı belirler (3.5). Örnek: `int m[3][4]` için `&m[2][1]` = taban + (2 × 4 + 1) × 4 = taban + **36**.

Uzaklıkları `(char *)` ile ölçtük: bütün matris **tek bir nesnedir** ve baytlarını `char` olarak saymak her zaman tanımlıdır; standart ise farklı satır dizilerine ait iki `int` adresinin farkını tanımlamaz. Aynı nedenle sonraki satıra "taşan" `m[0][4]` bellekte doğru yere düşse bile **tanımsız davranıştır** (Alıştırma 11.3'te UBSan yakalıyor).

3.5 İki boyutlu diziyi fonksiyona aktarmak

Dizi fonksiyona yine **adresiyle** gider (kopyalanmaz). Fonksiyonun `m[i][j]`'yi bulması için 3.4'teki formülü, yani **sütun sayısını** bilmesi gerekir: ilk boyut boş kalabilir (satır sayısı ayrı parametredir) ama **ikinci boyut parametre tipinde yazılmak zorundadır**: `int m[][SUTUN]`. Örnek 5'te `int ana_kosegen(int m[][N])` tanımlanır, `ana_kosegen(a)` ile çağrılır. Sütun boyutu unutulursa (`int m[][]`, Hata Avcısı 3) GCC durur: "array type has incomplete element type 'int[]'". Kural her boyuta genellenir: `int k[][3][2]` gibi parametrede de yalnızca ilk boyut boş kalabilir.

Bir satır 1B dizidir, bir sütun değildir. `m[i]` bellekte bitişik SUTUN elemanlı bir dizidir ve H10'un 1B fonksiyonlarına doğrudan verilir: `dizi_toplam(m[i], SUTUN)`. Bir sütunun ardışık elemanları arasında SUTUN × `sizeof(int)` bayt vardır; 1B fonksiyona vermek için önce geçici diziye **kopyalarız**: `sutun_kopyala` her `i` için `hedef[i] = m[i][j]` yapar (`H11_K03_satir_fonksiyona.c`; Mini-OBS v0.11'de de aynısı):

H11_K03_satir_fonksiyona.c · satır 36–40

```
36      for (int i = 0; i < SATIR; i++) {
37          printf("%d. satırın toplamı: %d\n", i, dizi_toplam(m[i], SUTUN));
38      }
39      sutun_kopyala(m, SATIR, 2, sutun);
40      printf("2. sütunun toplamı (kopya ile): %d\n", dizi_toplam(sutun, SATIR));
```

Gerçek çıktı: satır toplamları 10, 26, 42; 2. sütunun toplamı (kopya ile) 21.

MERAK KUTUSU · **const 2B parametre ve VLA parametresi**

Salt okuyan fonksiyonun parametresini `const` yapmak iyi alışkanlıktır (H10). `H11_K04_const_parametre.c`'deki `double iz(const double m[][N])` **const** bir diziyle sorunsuzdur; ama dosyanın kopyasında dizinin `const`'u silinince GCC 13 C17 kipinde `-pedantic` ile uyarır: "invalid use of pointers to arrays with different qualifiers in ISO C before C2X [-Wpedantic]". C17'de `double (*)[N]` → `const double (*)[N]` dönüşümü uyumsuz sayılır; C23 dizi ve elemanını aynı niteleyicili sayarak bunu düzeltti (WG14 N2607; Uecker, 2020). Aynı kopya `-std=c2x` ile ve Clang 18 ile **uyarısız** derlendi; Mini-OBS v0.11 bu yüzden matris parametrelerinde `const` kullanmaz.

Sütun sayısı değişken olabilir mi? C99'dan beri boyutlar parametre listesinde **dizinin önüne** yazılırsa: `void yazdir(int satir, int sutun, int m[satir][sutun])` (`H11_K08_vla_parametre.c`, 2 × 3 ve 3 × 2 matrisleri aynı fonksiyonla yazdırır). Değişken uzunluklu dizi (VLA) C11/C17'de **isteğe bağlıdır**: GCC ve Clang destekler, MSVC desteklemez. Taşınabilir yol H12'nin dinamik belleğidir.

3.6 Komşuluk ve sınır denetimi

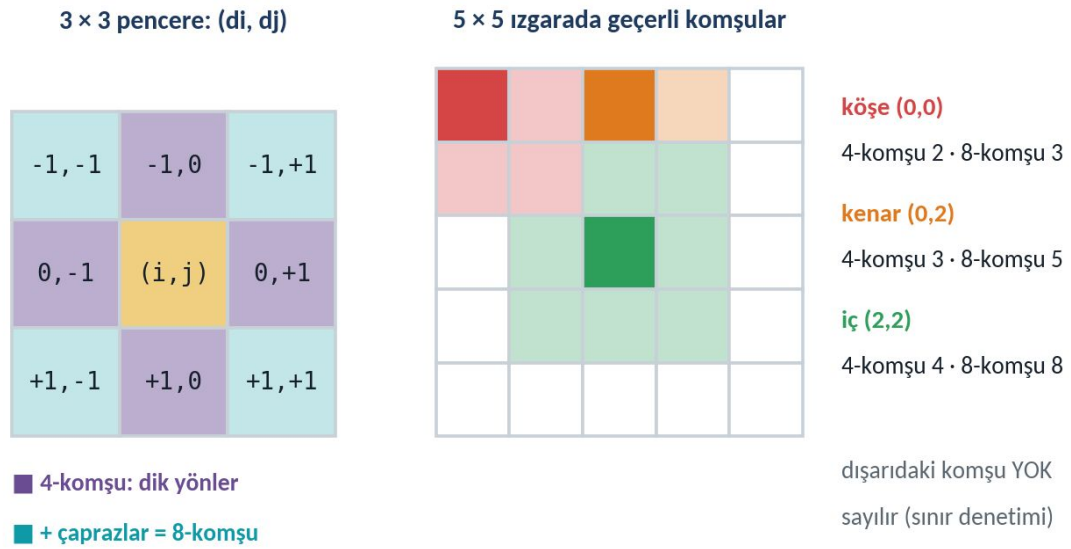
Izgara problemleri çoğu zaman bir hücrenin **komşularına** bakar (mayınlar, canlı komşular, çevre pikseller). **4-komşuluk** yukarı, aşağı, sol ve sağdır; dört çapraz eklenince **8-komşuluk** olur. Komşular iki döngüyle (`di, dj ∈ {-1, 0, 1}`, merkez hariç) ya da uzaklık dizileriyle dolaşılır (`H11_K07_komsuluk.c`):

H11_K07_komsuluk.c · satır 14–20

```
14  const int di[8] = {-1, 1, 0, 0, -1, -1, 1, 1};   /* ilk 4: dik yönler */
15  const int dj[8] = {0, 0, -1, 1, -1, 1, -1, 1};   /* son 4: çaprazlar */
16
17  int gecerli_komsu(int i, int j, int yon_sayisi)
18  {
19      int sayi = 0;
20      for (int k = 0; k < yon_sayisi; k++) {
```

Döngü `yon_sayisi` (4 ya da 8) kez döner; komşu `ni = i + di[k]`, `nj = j + dj[k]` konumundadır ve yalnızca `ni >= 0 && ni < SATIR && nj >= 0 && nj < SUTUN` ise sayılır. Kenar ve köşe hücrelerinde pencerenin bir kısmı **dışarı taşar** (Şekil 11.2); gerçek çıktı: “köşe (0,0): 4-komşu 2, 8-komşu 3”, “kenar (0,2): 4-komşu 3, 8-komşu 5”, “iç (2,2): 4-komşu 4, 8-komşu 8”. **Her komşu erişiminden önce** sınır denetlenmelidir; denetimsiz `m[i - 1][j]` köşede dizinin önündeki belleği okur (Hata Avcısı 2). `&&` kısa devresi (H5) sayesinde sınır dışı bir indisle `m[ni][nj]` hiç değerlendirilmez.

Şekil 11.2
3 × 3 komşuluk penceresi ve 5 × 5 ızgarada köşe, kenar ve iç hücrenin geçerli komşuları (H11_K07'nin gerçek çıktısı)



3.7 Görüntü = piksel matrisi

Gri tonlu görüntü `int g[YUK][GEN]` matrisidir; `g[y][x]` y. satırın x. pikselidir. Filtreler iç içe döngülerdir ve her biri **okuduğu matristen ayrı bir matrise** yazar (Şekil 11.3, Örnek 11): **negatif** `255 − p`; **eşikleme** `p >= 128 ? 255 : 0`; **yatay ayna** `s[i][j] = g[i][GEN − 1 − j]`; **3 × 3 ortalama bulanıklaştırma** (her piksel komşuluğunun ortalaması). Bulanıklaştırmada pencere kenarda taşar; bir **kenar politikası** seçilip belgelenmelidir. Bizimki: yalnızca görüntünün **içindeki** komşuların ortalaması (köşede 4, kenarda 6, içeride 9 piksel); başka seçenekler kenar pikselini kopyalamak ya da dışarıyı 0

saymaktır. Gerçek dosyalarda piksel genellikle `unsigned char` olur; renkli görüntü `[YUK][GEN][3]` **üç boyutlu** dizisidir.

Şekil 11.3
8 × 12 gri tonlu görüntü ve dört filtrenin sonucu (H11_11'in gerçek değerleri)



8 × 12 gri tonlu piksel matrisi (0 siyah … 255 beyaz); değerler H11_11'in gerçek çıktısından

Üç boyutlu dizilere kısa bakış

`int notlar[SUBE][OGRENCI][DERS]` iki şubenin her birinde üç öğrencinin iki dersteki notudur. Üç iç içe döngüyle dolaşılır; bellekte yine **tek şerittir** (önce A şubesinin bütün öğrencileri, sonra B'ninkiler) ve formül bir terim uzar: ((s × OGRENCI + o) × DERS + d) × `sizeof(int)` (`H11_K06_uc_boyutlu.c`):

H11_K06_uc_boyutlu.c · satır 17–20

```
17      int notlar[SUBE][OGRENCI][DERS] = {
18          {{70, 80}, {55, 65}, {90, 85}},        /* A şubesi */
19          {{60, 75}, {88, 92}, {45, 50}}         /* B şubesi */
20      };
```

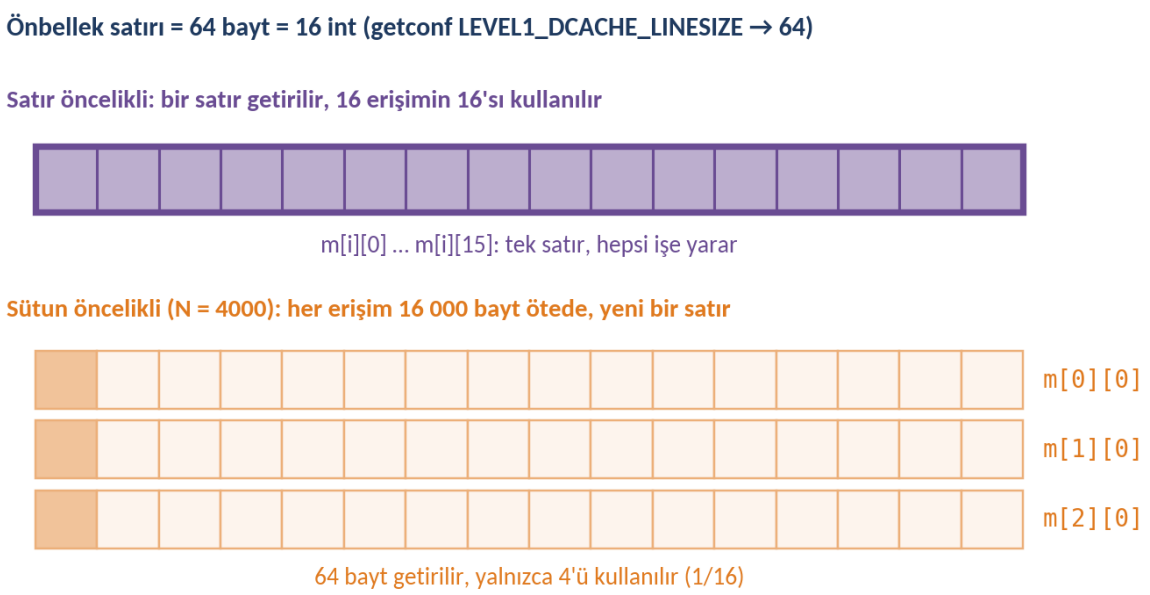
Gerçek çıktı: şube ortalamaları 74,17 ve 68,33; `sizeof notlar` = 48, `sizeof notlar[0]` = 24 (bir şube), `sizeof notlar[0][0]` = 8 (bir öğrenci); "uzaklık(notlar[1][2][1]) = 44 bayt, formül = 44".

3.8 Kaputun altında: önbellek ve dolaşma sırası

İşlemci her eleman için ana belleğe (DRAM) gitmez: arada küçük ama çok hızlı **önbellekler** (cache) vardır ve veri onlara **önbellek satırı** (cache line) denen bloklar hâlinde getirilir (Bryant & O'Hallaron, 2016, Bölüm 6; Patterson & Hennessy, 2017). Bu makinede `getconf LEVEL1_DCACHE_LINESIZE` **64 bayt** verdi: bir satır 16 `int` taşır. Ortam (`lscpu`, `getconf`): Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdek; çekirdek başına 32 KiB L1 veri ve 1 MiB L2, ortak 33 MiB L3 önbelleği.

Satır öncelikli dolaşmada getirilen 64 baytlık satır sonraki 15 erişimin verisini de taşır: **16 erişime bir getirme**. 4000 sütunlu matriste sütun öncelikli dolaşmanın ardışık iki erişimi **16 000 bayt** uzaktadır: her erişim yeni bir satır ister ve getirilen 64 baytın yalnızca 4'ü kullanılır (Şekil 11.4). Matris önbelleğe sığarsa satırlar sonraki sütunda yeniden kullanılabilir; ama 4000 × 4000 `int` **64 MB**'dır, 33 MiB'lık L3'e sığmaz. Bu, **veri yerelliği** (locality) ilkesidir.

Şekil 11.4
Önbellek satırı (64 bayt = 16 int): satır öncelikli dolaşma getirilen her baytı kullanır, sütun öncelikli dolaşma 1/16'sını



Deney (Örnek 12): 4000 × 4000 matrisin toplamı iki sırayla hesaplandı, süreler `clock()` ile ölçüldü. H7'nin ilkeleri: her ölçüm program içinde **7 kez** tekrarlandı ve **medyan** alındı; toplam her seferinde yazdırıldı (kullanılmayan sonucu iyileştirici silebilir); `-O0` (desteksiz derleme), `-O2` ve `-O3` ayrı ölçüldü; boyut da değiştirildi (`-DN=1000`, `-DN=2000`). Ortam **2 çekirdekli, paylaşımlı bir sanal makinedir**; mutlak sürelere değil **oranlara** bakın (Tablo 11.1).

Tablo 11.1
Önbellek deneyinin gerçek sonuçları (7 tekrarın medyanı; 2026-10-01, paylaşımlı sanal makine)

Derleme	N (bellek)	Satır öncelikli	Sütun öncelikli	ns / eleman (satır; sütun)	Sütun / satır
-O0	1 000 (4 MB)	2,5 ms	2,7 ms	2,50; 2,70	1,1×
-O0	2 000 (16 MB)	10,0 ms	16,7 ms	2,50; 4,17	1,7×
-O0	4 000 (64 MB)	40,6 ms	114,5 ms	2,54; 7,16	2,8×
-O2	2 000 (16 MB)	2,3 ms	17,0 ms	0,57; 4,25	7,4×
-O2	4 000 (64 MB)	8,5 ms	128,1 ms	0,53; 8,01	15,1×
-O3	4 000 (64 MB)	8,3 ms	8,3 ms	0,52; 0,52	1,0×

Yorum: (1) **-O0**'da N = 4000 için sütun öncelikli dolaşma **2,8 kat** yavaş; matris küçülünce fark azalır (N = 1000'de 1,1×). (2) **-O2** satır öncelikli döngüyü vektörleştirdi (`-fopt-info-loop`: "loop vectorized using 16 byte vectors", dört `int` tek komutla); sütun öncelikli döngü vektörleştirilemedi ve oran **15,1×**'e çıktı. (3) GCC 13 **-O2'de döngülerin sırasını değiştirmedi** (`-floop-interchange [disabled]`); **-O3**'te bu seçenek açıktır, derleyici "loops interchanged in loop nest" bildirdi ve fark kayboldu (1,0×). Ders: iyileştirici bazen kurtarır ama **buna güvenmeyin**; dolaşma sırasını veriye göre siz seçin.

4. Çözümlü Örnekler: Örnek Merdiveni

On dört örnek, her basamakta **bir yeni öge** ekler. Örnek 6 ve 8 standart çözümlü örnek şablonunun tamamıyla (problem, çözümleme, algoritma, kod, açıklama, izleme, çalıştırma, test) verilmiştir. Dosyalar `kodlar/` klasöründedir; çıktılar gerçek derleme ve çalıştırmadır (GCC 13.3, x86-64 Linux).

4.1 Örnek 1 (★): 3 × 3 matrisi okuyup hizalı yazdırmak

Problem. Kullanıcıdan 3 × 3 tamsayı matrisini satır satır okuyun ve sütunları hizalı biçimde yazdırın. **Yeni öge:** 2B bildirim, `&m[i][j]` ile `scanf` ve iç içe döngü.

H11_01_matris_oku_yaz.c · satır 16–27

```
16      int m[SATIR][SUTUN];                    /* 3 satır × 3 sütun = 9 int */
17
18      printf("%d x %d matrisi satır satır girin:\n", SATIR, SUTUN);
19      for (int i = 0; i < SATIR; i++) {       /* dış döngü: satır */
20          for (int j = 0; j < SUTUN; j++) {   /* iç döngü: sütun */
21              if (scanf("%d", &m[i][j]) != 1) {
22                  printf("Geçersiz girdi: m[%d][%d] okunamadı.\n",
23                         i, j);
24                  return 1;
25              }
26          }
27      }
```

ÇIKTI

```
$ ./matris_oku_yaz
3 x 3 matrisi satır satır girin:
4 -7 12
0 105 3
-18 6 9
Matris:
    4   -7   12
    0  105    3
  -18    6    9
```

`scanf` dokuz sayıyı satır sonlarından bağımsız okur. Yazdırma döngüsünün gövdesi `printf("%5d", m[i][j]);`, her satırın sonunda `printf("\n");`: `%5d` `-18` ile `105`'i aynı sütunda hizalar. Girdi `1 2 x` olunca program "Geçersiz girdi: m[0][2] okunamadı." der (ikinci çalıştırma, `ciktilar/` altında) (H6'dan beri `scanf`'in dönüş değeri denetlenir).

4.2 Örnek 2 (★): Satır ve sütun toplamları

Problem. Bir kantinin 3 kasasının 5 günlük satış adetleri tablodadır (`const int satis[KASA][GUN]`). Her kasanın haftalık toplamını (satır toplamı), her günün toplamını (sütun toplamı) ve genel toplamı yazdırın. **Yeni öge:** dolaşma yönü. Satır toplamları H7'nin tanıdık deseniyle (dış döngü `i`) hesaplanır; aşağıda yeni olan sütun toplamı kesiti var:

H11_02_satir_sutun_toplam.c · satır 33–43

```c
    int genel = 0;
    printf("Toplam ");
    for (int j = 0; j < GUN; j++) {          /* dış döngü: SÜTUN */
        int sutun_toplami = 0;
        for (int i = 0; i < KASA; i++) {     /* iç döngü: satır */
            sutun_toplami += satis[i][j];
        }
        printf("%5d", sutun_toplami);
        genel += sutun_toplami;
    }
    printf(" | %6d\n", genel);
```

ÇIKTI

```
$ ./satir_sutun_toplam
         Pzt  Sal  Çar  Per  Cum | Toplam
Kasa 1    12   15    9   20   18 |     74
Kasa 2     7   11   14   10   16 |     58
Kasa 3    21   19   17   25   22 |    104
Toplam    40   45   40   55   56 |    236
```

Sütun toplamında döngüler yer değiştirir: dış döngü `j` (gün), iç döngü `i` (kasa); toplayıcı her sütunda sıfırlanır. Denetim: satır toplamlarının toplamı (74 + 58 + 104) ile sütun toplamlarının toplamı aynı sayıyı (236) vermelidir.

4.3 Örnek 3 (★): Matris toplama

Problem. İki 2 × 3 matrisi toplayın. **Yeni öge:** eleman eleman işlem; aynı konumdaki elemanlar toplanır, `c[i][j] = a[i][j] + b[i][j]`. Toplama yalnızca **boyutları aynı** matrisler için tanımlıdır.

H11_03_matris_toplama.c · satır 19–22

```c
    for (int i = 0; i < SATIR; i++) {
        for (int j = 0; j < SUTUN; j++) {
            c[i][j] = a[i][j] + b[i][j];      /* aynı konumdaki elemanlar */
        }
```

ÇIKTI

```
$ ./matris_toplama
  1  2  3       10  20  30       11  22  33
  4  5  6   +   40  50  60   =   44  55  66
```

Çıkarma, sabitle çarpma ve eşitlik denetimi de aynı kalıptadır: iki döngü, gövdede tek işlem. `c = a + b;` yazılamaz (H10: diziler `=` ile kopyalanmaz).

4.4 Örnek 4 (★★): Devrik (transpoz) matris

Problem. 2 × 3 bir matrisin devriğini (satırların sütun olduğu 3 × 2 matris) yeni bir matrise yazın; ardından 3 × 3 bir matrisin devriğini **yerinde** alın. **Yeni öge:** indislerin yer değiştirmesi. Yeni matrise devrik tek satırdır: `int t[SUTUN][SATIR];` bildirilir ve iç içe döngünün gövdesi `t[j][i] = a[i][j];` olur. Yerinde devrik ise takas ister:

H11_04_devrik.c · satır 33–43

```
33      int k[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
34      int takas = 0;
35      for (int i = 0; i < N; i++) {
36          for (int j = i + 1; j < N; j++) {     /* yalnızca köşegenin üstü */
37              int gecici = k[i][j];
38              k[i][j] = k[j][i];
39              k[j][i] = gecici;
40              takas++;
41          }
42      }
43      printf("Yerinde devrik (3 x 3):\n");
```

ÇIKTININ YERİNDE DEVRİK BÖLÜMÜ (GERÇEK)

```
Yerinde devrik (3 x 3):
  1  4  7
  2  5  8
  3  6  9
Takas sayısı: 3 (N(N-1)/2)
```

Yerinde devrik **yalnızca kare** matriste mümkündür. `j = i + 1` kritiktir: köşegen kendisiyle takas edilmez, her (i, j) çifti **bir kez** işlenir; 3 takas = N(N − 1)/2. `j = 0`'dan başlasaydı her çift iki kez takas edilir ve matris değişmeden kalırdı (Alıştırma 11.4).

4.5 Örnek 5 (★★): Köşegenler, birim ve simetrik matris denetimi

Problem. 4 × 4 matrisler için ana ve yan köşegen toplamlarını hesaplayın; matrisin birim, simetrik ve üst üçgen olup olmadığını denetleyin. **Yeni öge:** koşullu dolaşma ve 2B dizinin fonksiyona aktarılması. Koşullar: ana köşegen `i == j`, yan köşegen `i + j == N - 1`, üst üçgen `j > i`, alt üçgen `j < i`.

H11_05_kosegen_simetri.c · satır 44–54

```
44  bool simetrik_mi(int m[][N])
45  {
46      for (int i = 0; i < N; i++) {
47          for (int j = i + 1; j < N; j++) {     /* yalnızca üst üçgen */
48              if (m[i][j] != m[j][i]) {
49                  return false;
50              }
51          }
52      }
53      return true;
54  }
```

ÇIKTI

```
$ ./kosegen_simetri
   Ana  Yan  Birim    Simetrik   Üst üçgen
A    10   34  hayır     evet       hayır
B     4    0  evet      evet       evet
C    17   10  hayır     hayır      evet
D    34   34  hayır     hayır      hayır
```

Köşegen için tek döngü yeter: `m[i][i]` (ana), `m[i][N − 1 − i]` (yan). Denetimler ilk **aykırılıkta** `false` döndürür (H10'un erken çıkış deseni); simetri yalnızca üst üçgeni gezer. B (birim) aynı zamanda simetrik ve üst üçgendir.

4.6 Örnek 6 (★★): Öğrenci × ders not tablosu (tam şablon)

1. Problem. Beş öğrencinin dört dersteki (MAT, FİZ, KİM, PRG) notları bir tabloda. Her öğrencinin ortalamasını, her dersin ortalamasını, her dersin en yüksek notunu ve sahibini, genel ortalamayı yazdırın. **Yeni öge:** satır ve sütun ortalamalarıyla **gerçek bir veri modeli** (satır = öğrenci, sütun = ders); Mini-OBS v0.11'in küçük bir provası.

Tablo 11.2
Örnek 6'nın girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• `notlar[5][4]` (sabit ilk değerler) • satır: öğrenci, sütun: ders	• satır toplamı / DERS • sütun toplamı / OGRENCI • sütunda en büyüğün satırı • genel toplam / 20	• hizalı tablo + öğrenci ortalamaları • ders ortalamaları ve en yüksekler • genel ortalama

3. Algoritma (sözde kod). her satır i için: toplam ← 0; her sütun j için toplam ← toplam + notlar[i][j]; satırı ve toplam / DERS'i yaz; genel ← genel + toplam. Sonra her sütun j için: sütun toplamı ve ortalaması; en ← 0, i = 1..OGRENCI − 1 için notlar[i][j] > notlar[en][j] ise en ← i; yaz notlar[en][j] ve en + 1.

H11_06_not_tablosu.c · satır 45–55

```
45      printf("\nEn yük.");
46      for (int j = 0; j < DERS; j++) {
47          int enb = 0;                          /* en yüksek notun satırı */
48          for (int i = 1; i < OGRENCI; i++) {
49              if (notlar[i][j] > notlar[enb][j]) {
50                  enb = i;
51              }
52          }
53          printf(" %2d(%d)", notlar[enb][j], enb + 1);
54      }
55      printf("\nGenel ortalama: %.2f\n",
```

5. Açıklama. Satır döngüsü (Örnek 2'deki gibi, `i` dışta) her öğrencinin notlarını ve ortalamasını yazar, `genel_toplam`'ı biriktirir. Ortalamalar `(double)` ile hesaplanır (H4). En yüksek not için H10'un **konum takibi** deseni bir **sütunda** uygulanır: `enb` bir satır indisidir, sütun `j` sabittir; eşitlikte ilk öğrenci kalır (`>`).

Tablo 11.3
İzleme tablosu: MAT sütununda (j = 0) en yüksek notun satırı

Adım (i)	notlar[i][0]	notlar[enb][0]	büyük mü?	enb
başlangıç	—	—	—	0
1	55	78	hayır	0
2	92	78	evet	**2**
3	67	92	hayır	2
4	84	92	hayır	2

7. Örnek çalıştırma.

ÇIKTI

```
$ ./not_tablosu
Öğrenci   MAT   FİZ   KİM   PRG    Ort.
      1    78    85    62    90   78.75
      2    55    70    48    66   59.75
      3    92    88    95    81   89.00
      4    67    59    73    70   67.25
      5    84    91    77    87   84.75
Ort.     75.2  78.6  71.0  78.8
En yük. 92(3) 91(5) 95(3) 90(1)
Genel ortalama: 75.90
```

8. Test. Elle hesapla karşılaştırma: 3. öğrencinin ortalaması (92 + 88 + 95 + 81) / 4 = 89,00; KİM ortalaması (62 + 48 + 95 + 73 + 77) / 5 = 71,0; genel ortalama 1518 / 20 = 75,90. Üçü de çıktıdaki değerlerle aynı.

9. Değiştir–genişlet. (a) Her dersten 60 ve üzeri alan öğrenci sayısı (sütunda sayma). (b) Ortalaması en yüksek öğrenci (satır ortalamalarında konum takibi). (c) Sabit `OGRENCI` bir sınırdır: Mini-OBS'te `AZAMI_OGRENCI` kapasite, `n` dolu satır sayısıdır; çalışma zamanında boyut H12'de.

4.7 Örnek 7 (★★): Sinema salonu koltuk rezervasyonu

Problem. 5 sıra × 8 koltukluk salonda kullanıcı sıra ve koltuk numarası (1'den başlayarak) girer; boş koltuk ayrılır, dolu ya da var olmayan koltuk reddedilir; `0 0` girişi bitirir, sonra salon planı yazdırılır. **Yeni öge: durum matrisi** (0 boş, 1 dolu) ve kullanıcı etkileşimi.

H11_07_sinema_koltuk.c · satır 46–54

```
46          if (s < 1 || s > SIRA || k < 1 || k > KOLTUK) {  /* sınır denetimi */
47              printf("  Böyle bir koltuk yok (sıra 1-%d, koltuk 1-%d).\n",
48                     SIRA, KOLTUK);
49          } else if (salon[s - 1][k - 1] == 1) {  /* 1 tabanlı → 0 tabanlı */
50              printf("  Sıra %d koltuk %d DOLU.\n", s, k);
51          } else {
52              salon[s - 1][k - 1] = 1;
53              dolu++;
54              printf("  Ayrıldı: sıra %d, koltuk %d.\n", s, k);
```

ÇIKTI (GERÇEK; KESİT)

```
$ ./sinema_koltuk
Sıra ve koltuk (çıkış: 0 0): 3 4
  Ayrıldı: sıra 3, koltuk 4.
Sıra ve koltuk (çıkış: 0 0): 3 4
  Sıra 3 koltuk 4 DOLU.
Sıra ve koltuk (çıkış: 0 0): 6 2
  Böyle bir koltuk yok (sıra 1-5, koltuk 1-8).
…
Dolu: 2 / 40
```

Salon `int salon[SIRA][KOLTUK] = {0};` ile boş başlar; `for (;;)` döngüsü `scanf` başarısız olunca ya da `0 0` ile `break` eder. Kullanıcı **1'den**, C **0'dan** sayar (`salon[s - 1][k - 1]`). Aralık denetimi erişimden **önce** yapılır: denetimsiz `6 2` `salon[5][1]`'e, yani dizinin dışına yazardı. Çıktı test durumlarını içerir: boş koltuk, aynı koltuk, aralık dışı sıra, son koltuk ve `0 0`; ikinci çalıştırmada Ctrl+D ile "Girdi sona erdi." yazılır, plan yine basılır (`ciktilar/` altında). `plan_yazdir` matrisi `int salon[][KOLTUK]` olarak alır.

4.8 Örnek 8 (★★★): Matris çarpımı (tam şablon)

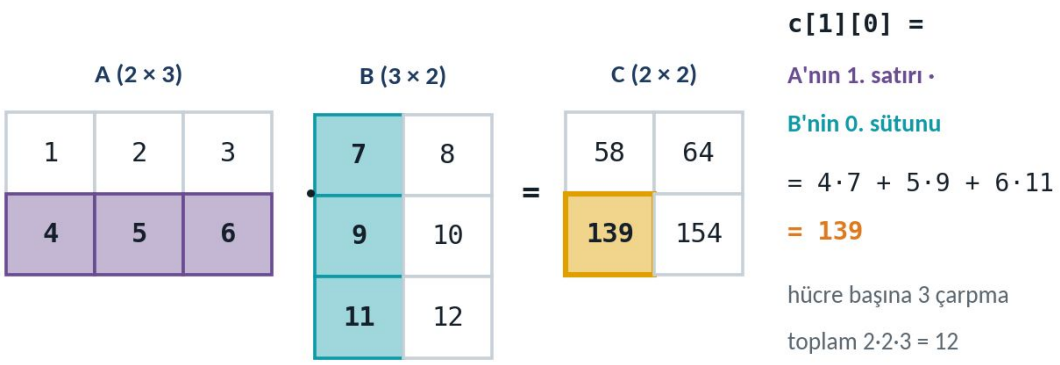
1. Problem. (n × m) bir A ile (m × p) bir B matrisinin çarpımı C'yi (n × p) hesaplayın ve yapılan çarpma sayısını sayın; kare matrislerde bu sayının n^3 olduğunu doğrulayın. **Yeni öge: üçlü iç içe döngü** ve n^3 işlem sezgisi.

Tablo 11.4
Örnek 8'in girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• A: n × m, B: m × p • koşul: A'nın sütun sayısı = B'nin satır sayısı	• $c[i][j] = \Sigma_k\, a[i][k] \cdot b[k][j]$ • her (i, j) için k = 0 … m − 1 • çarpma sayacı	• C: n × p • çarpma sayısı = n · m · p

3. Algoritma. C'nin her hücresi, A'nın **i. satırı** ile B'nin **j. sütununun** iç çarpımıdır (Şekil 11.5): i her satır için, j her sütun için: c[i][j] ← 0; k = 0 … m − 1 için c[i][j] ← c[i][j] + a[i][k] · b[k][j].

Şekil 11.5
Satır × sütun: c[1][0], A'nın 1. satırı ile B'nin 0. sütununun iç çarpımıdır (H11_08'in gerçek sonucu)



H11_08_matris_carpim.c · satır 14–28

```c
long long carp(int n, int m, int p, int a[][AZAMI], int b[][AZAMI],
               int c[][AZAMI])
{
    long long carpma = 0;                    /* yapılan çarpma sayısı */
    for (int i = 0; i < n; i++) {            /* a'nın satırı */
        for (int j = 0; j < p; j++) {        /* b'nin sütunu */
            c[i][j] = 0;                     /* ÖNCE sıfırla */
            for (int k = 0; k < m; k++) {
                c[i][j] += a[i][k] * b[k][j];
                carpma++;
            }
        }
    }
    return carpma;
}
```

5. Açıklama. Fonksiyon matrisleri `int x[][AZAMI]` olarak alır: bellekteki satır uzunluğu AZAMI'dir, kullanılan bölüm n, m, p ile belirlenir. `main`'de matrisler `static int a[AZAMI][AZAMI] = {{1, 2, 3}, {4, 5, 6}};` biçiminde (≈ 300 KB yığın yerine veri bölgesinde, H9; kısmi ilk değer kalanları 0 yapar). `c[i][j] = 0;` **unutulursa** `c` önceki içeriğin üstüne ekler (Hata Avcısı 4).

Tablo 11.5
İzleme tablosu: i = 1, j = 0 (sayaç önceki 2 hücreden 6'dır)

k	a[i][k]	b[k][j]	işlem	c[1][0]	sayaç
0	a[1][0] = 4	b[0][0] = 7	0 + 4·7	**28**	7
1	a[1][1] = 5	b[1][0] = 9	28 + 5·9	**73**	8
2	a[1][2] = 6	b[2][0] = 11	73 + 6·11	**139**	9

7. Örnek çalıştırma.

ÇIKTI

```
$ ./matris_carpim
(2 × 3) · (3 × 2) = 2 × 2:
    58    64
   139   154
çarpma sayısı: 12 (2 · 3 · 2)

     n      çarpma          n³
    10        1000        1000
    20        8000        8000
    40       64000       64000
    80      512000      512000
   160     4096000     4096000
```

8. Test. Küçük örnekte elle hesap (58, 64, 139, 154) ile çıktı aynıdır ve çarpma sayısı 2 · 3 · 2 = 12'dir. Kare matrislerde sayaç her satırda n^3'e eşittir: n iki katına çıkınca çarpma sayısı **8 katına** çıkar (1 000 → 8 000 → 64 000 …). AP-2'de "kübik" büyüme olarak adlandırılacak bu artış 1000 × 1000 için bir milyar çarpma demektir.

9. Değiştir–genişlet. (a) A'nın sütun sayısı B'nin satır sayısına eşit değilse çarpımı reddedin. (b) `A · I = A` olduğunu doğrulayın. (c) `A · B` ile `B · A`'yı karşılaştırın: çarpım değişmeli değildir.

4.9 Örnek 9 (★★★): Tic-tac-toe ve kazanan denetimi

Problem. İki oyuncu 3 × 3 tahtada sırayla hamle girer; geçersiz hamle (aralık dışı ya da dolu hücre) tekrarlatılır; her hamleden sonra kazanan aranır; dokuz hamlede kazanan yoksa berabere. **Yeni öge:** satır, sütun ve iki köşegen olmak üzere **8 çizginin** denetimi.

H11_09_tictactoe.c · satır 33–40

```
33      for (int i = 0; i < N; i++) {
34          if (ayni(t[i][0], t[i][1], t[i][2])) {         /* i. satır */
35              return t[i][0];
36          }
37          if (ayni(t[0][i], t[1][i], t[2][i])) {         /* i. sütun */
38              return t[0][i];
39          }
40      }
```

ÇIKTI (GERÇEK, SON SATIRLAR)

```
…
 O | . | X
---+---+---
 O | X | .
---+---+---
 X | . | .
Kazanan: X (5 hamlede)
```

Tek döngü (`i`) aynı turda i. satırı (`t[i][0..2]`) ve i. sütunu (`t[0..2][i]`) denetler; ardından iki köşegen (`t[0][0], t[1][1], t[2][2]` ve `t[0][2], t[1][1], t[2][0]`) aynı biçimde denetlenir, kazanan yoksa `'.'` döner. `ayni` boş hücreyi (`'.'`) kazanan saymaz. `main`'deki hamle döngüsü önce `s < 1 || s > N || k < 1 || k > N || t[s - 1][k - 1] != '.'` denetimini yapar (kısa devre, dizi dışı okumayı engeller); geçersiz hamlede `continue` aynı oyuncuyu yeniden sorar. İkinci çalıştırma "Berabere." ile biter.

4.10 Örnek 10 (★★★): Mayın tarlası komşu sayıları

Problem. 6 × 8 mayın haritasında (1 mayın, 0 boş) her boş hücre için çevresindeki 8 hücredeki mayın sayısını yazdırın. **Yeni öge:** 8-komşuluk ve **sınır denetimi** (pencere için bkz. Şekil 11.2).

H11_10_mayin_tarlasi.c · satır 18–29

```
18      for (int di = -1; di <= 1; di++) {
19          for (int dj = -1; dj <= 1; dj++) {
20              int ni = i + di, nj = j + dj;      /* komşunun konumu */
21              if (di == 0 && dj == 0) {
22                  continue;                      /* hücrenin kendisi */
23              }
24              if (ni >= 0 && ni < SATIR && nj >= 0 && nj < SUTUN) {
25                  sayi += m[ni][nj];             /* yalnızca ızgara içi */
26              }
27          }
28      }
29      return sayi;
```

ÇIKTI

```
$ ./mayin_tarlasi
 * 2 1 1 . . 1 *
 1 2 * 2 2 2 2 1
 1 2 3 3 * * 2 .
 1 * 2 * 4 * 2 .
 1 1 2 1 2 1 2 1
 . . . . . . 1 *
Mayın: 9, en kalabalık hücre: 4 komşu mayın
```

`(0, 0)` hücrenin kendisidir, atlanır. Kenarda `ni = -1` ya da `nj = SUTUN` gibi konumlar oluşur; `if` bunları erişimden önce eler. `printf(" %c", '0' + k)` sayıyı rakam karakterine çevirir (H3).

4.11 Örnek 11 (★★★): Görüntü işleme: negatif, eşik, ayna, bulanıklaştırma

Problem. 8 × 12 gri tonlu bir görüntüye (sağa bakan parlak bir ok, soldan sağa koyulaşan bir arka plan) dört filtre uygulayın ve sonuçları ASCII sanatıyla çizin. **Yeni öge:** piksel matrisi, **ayrı çıktı matrisi** ve bulanıklaştırmada kenar politikası (Şekil 11.3).

H11_11_goruntu_isleme.c · satır 69–86

```
69  void bulaniklastir(int g[][GEN], int s[][GEN])
70  {
71      for (int i = 0; i < YUK; i++) {
72          for (int j = 0; j < GEN; j++) {
73              int toplam = 0, adet = 0;
74              for (int di = -1; di <= 1; di++) {
75                  for (int dj = -1; dj <= 1; dj++) {
76                      int ni = i + di, nj = j + dj;
77                      if (ni >= 0 && ni < YUK && nj >= 0 && nj < GEN) {
78                          toplam += g[ni][nj];
79                          adet++;               /* köşe 4, kenar 6, iç 9 */
80                      }
81                  }
82              }
83              s[i][j] = (toplam + adet / 2) / adet;   /* yuvarlanmış ort. */
84          }
85      }
86  }
```

ÇIKTININ BULANIK BÖLÜMÜ (GERÇEK)

```
3 × 3 ortalama bulanıklaştırma
    ......==++**==----==|
    ......==**##++==--==|
..::----==++##%%##++====|
--==******##%%%%%%**++==|
--==******##%%%%%%**++==|
..::----==++##%%##++====|
    ......==**##++==--==|
    ......==++**==----==|
```

ASCII sanatında piksel değeri 10 düzeyli bir **tona** çevrilir: `ton[p * 10 / 256]` (dizi bir arama tablosu); her piksel iki karakterle basılır, çünkü karakterler yüksekliklerinin yaklaşık yarısı genişliktedir. Bulanıklaştırma `g`'ye **asla yazmaz**, sonucu `s`'ye koyar; yerinde yapsaydı sağdaki komşu soldakinin **yeni** değerini okurdu (Hayat Oyunu'nda aynı hata). Keskin kenar sayılarda da yumuşar: (1,5) pikseli 55'tir, dokuz komşusunun toplamı 993'tür, bulanık değeri (993 + 4) / 9 = **110** olur.

4.12 Örnek 12 (★★★): Satır ve sütun öncelikli dolaşma süre deneyi

Problem. 4000 × 4000 `int` matrisin toplamını önce satır öncelikli, sonra sütun öncelikli dolaşarak hesaplayın; iki sürenin oranını ölçün. **Yeni öge:** önbellek etkisinin deneysel gözlemi (kavram ve bütün sonuçlar Bölüm 3.8'de).

H11_12_onbellek_sure.c · satır 33–42

```
33  long long sutun_oncelikli(int a[][N])
34  {
35      long long toplam = 0;
36      for (int j = 0; j < N; j++) {
37          for (int i = 0; i < N; i++) {
38              toplam += a[i][j];                /* adım: N · 4 bayt */
39          }
40      }
41      return toplam;
42  }
```

`main` iki çağrıyı `clock()` okumalarının (`bas`, `ara`, `son`) arasına koyar ve her yöntemin en kısa süresini tutar (H7); çıktısı her tekrarın iki süresini, toplamları ve oranı verir (`ciktilar/H11_12_onbellek_sure.txt`; özet Tablo 11.1). `static int m[N][N];` **64 MB**'dır: yerel olsaydı 8 MiB'lık yığını (H9) aşardı. İki toplam aynıdır ("toplamlar eşit: evet"), yalnızca **sıra** farklıdır.

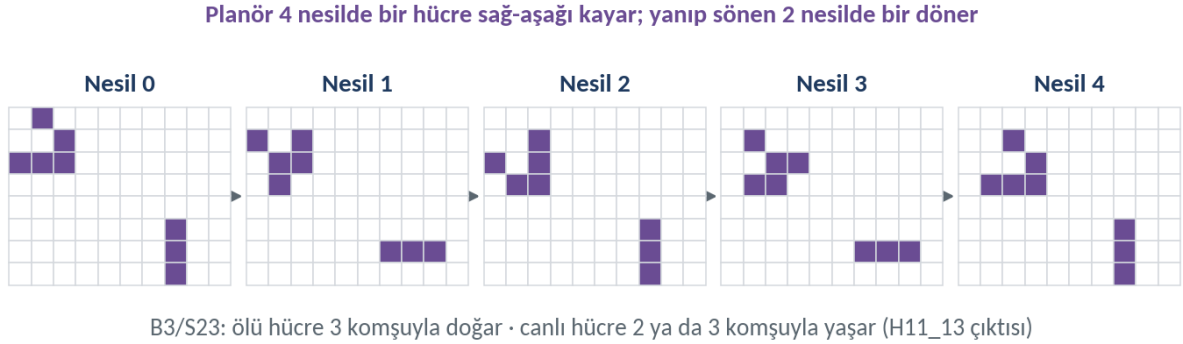
4.13 Örnek 13 (★★★): Conway'in Hayat Oyunu (meydan okuma)

Problem. 8 × 10 ızgarada bir planör ve bir yanıp sönen (blinker) ile başlayıp dört nesil hesaplayın; ızgaranın dışı ölü sayılır. **Yeni öge:** nesil hesaplama ve **iki tampon** matris (Şekil 11.6).

H11_13_hayat_oyunu.c · satır 34–46

```
34  void sonraki_nesil(int eski[][SUTUN], int yeni[][SUTUN])
35  {
36      for (int i = 0; i < SATIR; i++) {
37          for (int j = 0; j < SUTUN; j++) {
38              int k = canli_komsu(eski, i, j);
39              if (eski[i][j] == 1) {
40                  yeni[i][j] = (k == 2 || k == 3);   /* S23: yaşar */
41              } else {
42                  yeni[i][j] = (k == 3);             /* B3: doğar */
43              }
44          }
45      }
46  }
```

Şekil 11.6
Hayat Oyunu'nun beş nesli (H11_13'ün gerçek çıktısından çizildi; metin hâli kodlar/ciktilar/H11_13_hayat_oyunu.txt): planör nesil 4'te bir hücre sağ-aşağı kaymış, yanıp sönen başa dönmüş



Püf noktası **eşzamanlılıktır**: her hücrenin yeni durumu bütün hücrelerin **eski** durumundan hesaplanır. Bu yüzden `sonraki_nesil eski`'den okur, `yeni`'ye yazar; nesil bitince `yeni g`'ye kopyalanır (tek matrisle: Hata Avcısı 5). Planörün nesil 0'daki hücreleri (0,1), (1,2), (2,0), (2,1), (2,2); nesil 4'te (1,2), (2,3), (3,1), (3,2), (3,3): hepsi (+1, +1) kaymış, Gardner'ın (1970) betimlediği gibi.

4.14 Örnek 14 (★★★): Spiral matris (meydan okuma)

Problem. Bir matrisi 1, 2, 3, … sayılarıyla dıştan içe saat yönünde **spiral** sırada doldurun; ayrıca satır öncelikli doldurulmuş matrisleri spiral sırada okuyun (3 × 5, 1 × 4, 5 × 1). **Yeni öge:** dört sınırın yönetimi. `spiral_doldur` başta `ust = 0, alt = satir - 1, sol = 0, sag = sutun - 1` ve `sayi = 1` ile başlar:

H11_14_spiral.c · satır 20–34

```
20      while (ust <= alt && sol <= sag) {
21          for (int j = sol; j <= sag; j++) {    /* üst satır: soldan sağa */
22              m[ust][j] = sayi++;
23          }
24          ust++;
25          for (int i = ust; i <= alt; i++) {    /* sağ sütun: yukarıdan aşağı */
26              m[i][sag] = sayi++;
27          }
28          sag--;
29          if (ust <= alt) {                     /* alt satır kaldı mı? */
30              for (int j = sag; j >= sol; j--) {
31                  m[alt][j] = sayi++;
32              }
33              alt--;
34          }
```

ÇIKTI

```
$ ./spiral
4 × 5 spiral doldurma:
   1   2   3   4   5
  14  15  16  17   6
  13  20  19  18   7
  12  11  10   9   8
3 × 5 spiral okuma: 1 2 3 4 5 10 15 14 13 12 11 6 7 8 9
1 × 4 spiral okuma: 1 2 3 4
5 × 1 spiral okuma: 1 2 3 4 5
```

Değişmez (invariant): işlenmemiş bölge her zaman `[ust..alt] × [sol..sag]` dikdörtgenidir; her kenardan sonra o sınır içeri çekilir. Kesitin devamında sol sütun `if (sol <= sag)` denetimiyle doldurulur ve `sol++` yapılır. 3 × 5'te son turda tek satır, 5 × 1'de tek sütun kalır; bu `if`'ler olmasaydı aynı hücreler ikinci kez işlenirdi. Çıktının son üç satırı bu sınır durumlarının testidir.

5. Hata Avcısı

İki boyutlu dizilerin hataları çoğu zaman **derleyicinin göremediği** yerdedir; bu yüzden bellek örneklerini `-fsanitize=address,undefined` ile de çalıştırıyoruz. Altı hata `kodlar/hata_avcisi/` altında hatalı (başında "BU DOSYA KASITLI HATA İÇERİR") ve düzeltilmiş sürümleriyle ve gerçek iletileriyle bulunur (Tablo 11.6).

Tablo 11.6
Bu haftanın Hata Avcısı özeti (gerçek iletiler)

#	Hata	Derleyici / araç iletisi ya da belirti	Çözüm
1	`[i][j]` yerine `[j][i]`	uyarı yok; çöp ortalamalar; ASan: AddressSanitizer: stack-buffer-overflow	`notlar[i][j]` (bildirim sırası)
2	Kenarda sınır denetimsiz komşu	uyarı yok; ASan: AddressSanitizer: stack-buffer-underflow	erişimden önce `0 <= ni < SATIR …`
3	Parametrede sütun boyutu yok	error: array type has incomplete element type 'int[]'	`int m[][SUTUN]`
4	Çarpımda sonuç sıfırlanmamış	uyarı yok; ikinci çarpım yanlış (A × I ≠ A)	`c[i][j] = 0;` k döngüsünden önce
5	Hayat Oyunu'nda tek tampon	uyarı yok; yanıp sönen tek nesilde ölüyor	eskiden oku, yeniye yaz
6	İlk değerde iç parantez yok	warning: missing braces around initializer [-Wmissing-braces]	`{{1, 2}, {4, 5}}`

Hata 1: [i][j] yerine [j][i]

hata_avcisi/H11_HA1_ij_karisik.c · satır 18–21

```
18      for (int i = 0; i < OGRENCI; i++) {
19          int toplam = 0;
20          for (int j = 0; j < DERS; j++) {
21              toplam += notlar[j][i];          /* HATA: [i][j] olmalı */
```

GERÇEK ÇIKTI (İKİ ÇALIŞTIRMA VE ASAN RAPORUNDAN SATIRLAR)

```
$ ./ha1      # 1. çalıştırma
Öğrenci 1 ortalaması: 377556008.00
Öğrenci 2 ortalaması: 585199963.67

$ gcc -std=c17 -g -fsanitize=address H11_HA1_ij_karisik.c -o ha1_san && ./ha1_san
==10388==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f5242800038 at
pc 0x559812b554da bp 0x7ffc569a88e0 sp 0x7ffc569a88d0
    #0 0x559812b554d9 in main H11_HA1_ij_karisik.c:21
    #0 0x559812b55258 in main H11_HA1_ij_karisik.c:15
```

Dizi 2 × 3'tür ama `notlar[j][i]`'de ilk indis **2**'ye kadar çıkar: `notlar[2][…]` dizinin dışındadır. Okunan çöp değer çalıştırmadan çalıştırmaya değişir (tanımsız davranış). Kare matriste aynı hata dizi dışına çıkmaz ama satır yerine **sütun** ortalamasını verir: sessizce yanlış sonuç. Kural: bildirim `[OGRENCI][DERS]` ise erişim de `[öğrenci][ders]`.

Hata 2: kenar hücrelerinde sınır denetimsiz komşu

hata_avcisi/H11_HA2_kenar_komsu.c · satır 19–20

```
19              if (di != 0 || dj != 0) {
20                  sayi += m[i + di][j + dj];    /* HATA: sınır denetimi yok */
```

GERÇEK ÇIKTI (HATALI SÜRÜM VE ASAN)

```
$ ./ha2
 -1425438636 54652 32769 5 5
 2 2 0 2 2
 2 2 2 2 -1080098238
 -1080065476 -598411971 -900590494 -1980721563 752526755

$ gcc -std=c17 -g -fsanitize=address H11_HA2_kenar_komsu.c -o ha2_san && ./ha2_san
==10414==ERROR: AddressSanitizer: stack-buffer-underflow on address 0x7ffb9dd00018 at
pc 0x557a03188315 bp 0x7fff5f30d9e0 sp 0x7fff5f30d9d0
    #0 0x557a03188314 in komsu_mayin H11_HA2_kenar_komsu.c:20
    #0 0x557a0318834e in main H11_HA2_kenar_komsu.c:28
```

Yalnızca iç hücrelerin sonuçları doğrudur; kenarlar dizinin önündeki ve arkasındaki belleği okur. ASan hatanın satırını (`komsu_mayin`, satır 20) ve adresin `mayin` dizisinin **önünde** olduğunu ("underflows this variable") söyler. Düzeltme Örnek 10'daki desendir.

Hata 3: parametrede sütun boyutu yok

hata_avcisi/H11_HA3_sutun_boyutu_yok.c · satır 10

```
10  void yazdir(int m[][], int satir, int sutun)
```

Açıklama: "declaration of 'm' as multidimensional array must have bounds for all dimensions except the first". Derleyici "i satır atla" diyebilmek için satırın kaç bayt olduğunu bilmelidir (3.4). Düzeltme: `void yazdir(int m[][SUTUN], int satir)`.

Hata 4: çarpımda sonuç matrisinin sıfırlanmaması

`hata_avcisi/H11_HA4_sonuc_sifirlanmadi.c · satır 17–18`

```
            for (int k = 0; k < N; k++) {     /* HATA: c[i][j] = 0 yok */
                c[i][j] += a[i][k] * b[k][j];
```

Program önce A × B'yi doğru yazar (19 22 / 43 50), çünkü `main` c'yi `{{0}}` ile başlatmıştı. İkinci çağrıda A × I, A'nın kendisi (1 2 / 3 4) olmalıydı; program önceki sonucu (19 22 / 43 50) ekleyerek 20 24 / 46 54 yazdı. H6'nın **toplayıcı deseni**: toplayıcı her yeni toplamdan önce sıfırlanır.

Hata 5: Hayat Oyunu'nda aynı matrisin hem okunması hem yazılması

`hata_avcisi/H11_HA5_hayat_tek_tampon.c · satır 43–46`

```
    for (int i = 0; i < N; i++) {             /* HATA: tek tampon */
        for (int j = 0; j < N; j++) {
            int k = canli_komsu(g, i, j);
            g[i][j] = g[i][j] ? (k == 2 || k == 3) : (k == 3);
```

Hatalı sürüm "1 nesil sonra:" altına beş satır `.....` yazar: en üst hücre (1,2) ölünce orta hücre artık tek canlı komşu görür ve o da ölür; doğması gereken (2,1) ve (2,3) yeterli komşu bulamaz. Düzeltilmiş sürüm (iki tampon) doğru biçimde `.###.` yazar.

Hata 6: ilk değer listesinde iç süslü parantezlerin unutulması

`hata_avcisi/H11_HA6_eksik_parantez.c · satır 13`

```
    int m[2][3] = {1, 2, 4, 5};               /* HATA: iç parantez yok */
```

GCC `-Wall` uyarır: "missing braces around initializer [-Wmissing-braces]". Niyet `{1, 2}` ve `{4, 5}` satırlarıydı; düz liste satır öncelikli doldurulur ve 4 birinci satıra kayar: program `1 2 4` ve `5 0 0` yazar. Düzeltme `{{1, 2}, {4, 5}}` (çıktı `1 2 0 / 4 5 0`).

6. Mühendis Gözüyle

- **Verimlilik: veri yerelliği.** Matrisi bellekteki sırasıyla (satır öncelikli) dolaşın: 64 MB'lık matriste ters sıra -O0'da 2,8, -O2'de 15,1 kat yavaştı; işlem saymak tek başına yetmez.
- **Güvenlik: sınır denetimi.** Dizi dışı erişimin en sık yeri kenarlardır (komşuluk, bulanıklaştırma); SEI CERT C'nin ARR30-C kuralı dizi dışı indisi yasaklar (Carnegie Mellon University Software Engineering Institute, 2016). Kullanıcıdan gelen 1 tabanlı koordinatları her zaman denetleyin; testleri sanitizer'la çalıştırın.
- **Test: sınır durumları.** 1 × N, N × 1, köşe hücresi, tam dolu ve tam boş ızgara hataların saklandığı yerlerdir. Satır/sütun karışıklığı kare matriste görünmez: **dikdörtgen** test verisi kullanın.
- **Okunabilirlik ve taşınabilirlik.** Boyutları `#define` ile adlandırın, `i` satır / `j` sütun alışkanlığını bozmayın, indis sırasını bildirime yorumla yazın. VLA parametreleri MSVC'de derlenmez; büyük matrisler yerel olursa yığın sınırı (Linux 8 MiB, MSVC 1 MB; H9) aşılır: `static` ya da (H12'de) dinamik bellek kullanın.

7. Mini-OBS Adımı: v0.11 — Öğrenci × Ders Not Matrisi

Gereksinim. v0.10'un tek dersli paralel dizilerini çoklu derse genişletin: notlar `[öğrenci][ders]` matrislerinde (`vize[i][d]`, `final[i][d]`, `ort[i][d]`; `DERS_SAYISI = 3`) tutulsun. Menü: **1) Ekle 2) Ders istatistikleri 3) Transkript 4) Harf tablosu 5) Listele 6) Çıkış**. Yeni çıktılar: her dersin ortalaması, standart sapması, en yüksek ve en düşük notu, başarılı ve şartlı sayısı; ders × harf **frekans matrisi**; bir öğrencinin **kredi ağırlıklı YANO**'su ve **transkripti**. Böylece v0.5'ten beri "YANO ≥ 2,00 ise" diye ertelediğimiz DC kararı artık hesaplanabilir (Şekil 11.7).

Dayanak. RTEÜ Ön Lisans ve Lisans Eğitim-Öğretim ve Sınav Yönetmeliği'nin (Recep Tayyip Erdoğan Üniversitesi, 2025) 31. maddesinin 2. fıkrasının (a) bendine göre not ortalaması, her dersin **kredisi** ile harf notu katsayısının çarpımlarının toplamının toplam krediye bölünmesiyle bulunur ve virgülden sonra iki haneye yuvarlanır (üçüncü hane 5 ve üzeriyse yukarı); `yano_hesapla` bu yuvarlamayı kayan nokta hatasına düşmeden tamsayıyla yapar: s = Σ kredi × (katsayı × 100), K = Σ kredi, yuvarlanmış YANO × 100 = (2s + K) / (2K) (49,5/17 = 2,9118 → 2,91). DC kararı bu yuvarlanmış değerle verilir. 4. maddeye göre kredi, dersin **ulusal kredisidir**. Katsayılar 30. maddenin 3. fıkrasındadır (AA 4,0 … FF 0,0); 30/4-b'ye göre YANO'su en az 2,00 olan öğrenci o yarıyıldaki DC notlarından da başarılı sayılır. Programdaki ders adları ve kredileri (6, 6, 5) **örnektir** (ders içi modelleme).

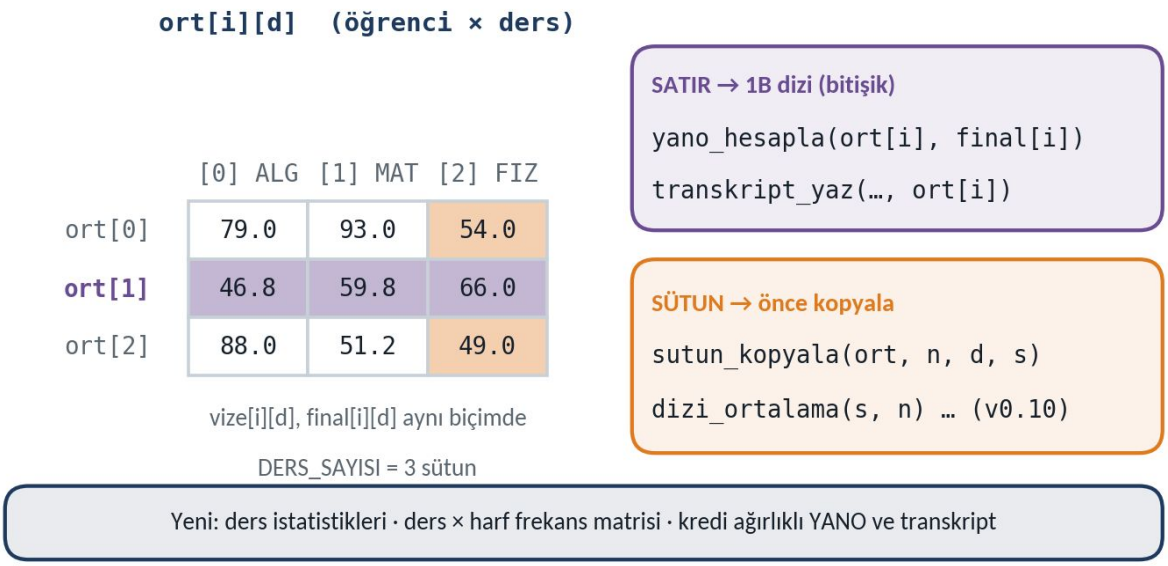
İpuçları. (1) Öğrenci eklerken `for (int d = 0; d < DERS_SAYISI; d++)` ile her dersin iki notu okunur ve `ort[n][d]` hesaplanır. (2) Bir öğrencinin satırı (`ort[i]`) H10'un 1B fonksiyonlarına **doğrudan** verilir. (3) Bir dersin sütunu bitişik değildir: `sutun_kopyala` ile geçici diziye alınır ve v0.10'un `dizi_ortalama`, `standart_sapma`, `en_buyuk_indis` fonksiyonları **yeniden yazılmadan** kullanılır:

proje/obs.c · satır 348–353

```
348  void sutun_kopyala(double m[][DERS_SAYISI], int n, int d, double hedef[])
349  {
350      for (int i = 0; i < n; i++) {
351          hedef[i] = m[i][d];
352      }
353  }
```

(4) `yano_hesapla(ort[i], final[i])` bir öğrencinin iki satırını alır ve her ders için kredi × (katsayı × 100) toplayıp sonucu iki haneye yuvarlar. (5) Ders × harf dağılımı H10'daki frekans dizisinin iki boyutlu hâlidir: `sayac[d][harf_indeksi(...)]++`. (6) Matris parametrelerinde `const` kullanılmadı (3.5, Merak Kutusu); 1B satır parametreleri `const`'tur. T1 senaryosunun transkripti, ders istatistikleri ve harf tablosu `91_Proje_MiniOBS/v0.11/mini_obs_v0_11.md` dosyasında gerçek çıktılarıyla verilmiştir.

Şekil 11.7
Mini-OBS v0.11 veri modeli: bir öğrencinin satırı 1B dizi olarak fonksiyonlara verilir, bir dersin sütunu önce kopyalanır



Kabul ölçütleri. Dört kaynak dosya sıfır uyarıyla derlenir; `make test` 61 doğrulamanın hepsini geçer (37 kural + 14 dizi + 10 matris; matris testleri YANO yuvarlamasının sınır durumunu da sınar); matris parametrelerinde sütun sayısı `DERS_SAYISI` ile yazılır; ders istatistikleri sütun kopyası ve 1B fonksiyonlarla hesaplanır; her ders satırının harf toplamı öğrenci sayısına eşittir; aynı numara iki kez eklenemez; girdi yarıda biterse "Girdi sona erdi." ile 1 döndürülür; ASan + UBSan temizdir.

Test senaryoları. `91_Proje_MiniOBS/v0.11/testler/` altındaki altı girdi dosyası (T1 üç öğrenci ve bütün raporlar; T2 geçersiz girdiler, tekrar eden ve bilinmeyen numara; T3 DC notu ile YANO < 2 ve ≥ 2; T4 not girişinde girdi sonu; T5 kayıt yokken raporlar; T6 yuvarlama ve final barajı sınırları) gerçek çıktılarıyla `mini_obs_v0_11_ciktilar.txt` dosyasındadır. Örneğin T1'de YANO'lar 2,91, 1,44, 2,06; T3'te YANO 1,47 ve 2,68 olan iki öğrencinin DC notları sırasıyla "başarısız sayılır" ve "başarılı sayılır"; T4'ün çıkış kodu 1.

8. Alıştırmalar ve Atölye

Atölye etkinlikleri ders saatinde yapılır; alıştırmaların kodları (11.1–11.6) `kodlar/alistirmalar/` klasöründedir; çözümler öğretim üyesindedir. ★ temel, ★★ orta, ★★★ ileri.

ALGORİTMA ATÖLYESİ · **Atölye etkinlikleri (75 dk)**

- **Matris kütüphanesi (30 dk, eşli).** `matris_oku`, `matris_yazdir`, `matris_topla`, `matris_devrik` ve `matris_carp` fonksiyonlarını `int m[][AZAMI]` ve ayrı satır/sütun parametreleriyle yazın (Örnek 8'in deseni); boyutlar uyumsuzsa 0 döndürün, devriği ayrı bir hedef matrise yazın. Test: A · I = A, (2 × 3) · (3 × 2) çarpımında Örnek 8'in sonucu, kare olmayan veri; `-fsanitize=address,undefined` ile temiz.
- **İki oyunculu tic-tac-toe (25 dk).** Örnek 9'u genişletin: tahta her hamleden sonra çizilsin; harf, aralık dışı ya da dolu hücre girişi reddedilsin; kazanan ya da beraberlik bildirilsin; oyun sonunda "yeniden oyna?" sorulsun. Satır, sütun ve iki köşegenle kazanılan dört oyun ve bir beraberlikle sınayın.
- **Görüntü filtreleri (20 dk).** Örnek 11'in görüntüsüne dikey ayna (`s[i][j] = g[YUK - 1 - i][j]`) ve parlaklık (+40, 255'i aşan değer 255'e kırpılır) filtrelerini ekleyin; bulanıklaştırmayı "kenarı kopyala" politikasıyla yeniden yazıp sonucu web sayfasındaki önizleyiciyle karşılaştırın.

A) Çıktı tahmini

Alıştırma 11.1 ★☆☆ *Çıktı tahmini*

Programı çalıştırmadan üç satırlık çıktıyı yazın.

H11_A01_cikti_tahmini.c · satır 8–25

```c
int main(void)
{
    int m[3][3];
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            m[i][j] = i * 3 + j + 1;
        }
    }
    for (int j = 0; j < 3; j++) {
        printf("%d ", m[2][j]);
    }
    printf("\n%d\n", m[1][2] + m[2][1]);
    for (int i = 0; i < 3; i++) {
        printf("%d ", m[i][2 - i]);
    }
    printf("\n");
    return 0;
}
```

Alıştırma 11.2 ★★☆ *Çıktı tahmini*

`H11_A02_adres_tahmini.c`, `double t[4][5]` için dört satır yazdırır (`sizeof(double)` = 8): (1) `sizeof t` ve `sizeof t[1]`; (2) `&t[2][3]`'ün, (3) `&t[3][0]`'ın `&t[0][0]`'dan bayt uzaklığı; (4) `&t[1][4]` ile `&t[1][0]` arasındaki bayt farkı. Çalıştırmadan hesaplayın, sonra programla doğrulayın.

B) Hata bulma

Alıştırma 11.3 ★★☆ *Hata bul*

Tüm elemanların toplamı 78 olmalı ama program her çalıştırmada başka bir sayı yazıyor. Hatayı bulun, nedenini açıklayın, düzeltin. Programı `-fsanitize=address,undefined` ile derleyip çalıştırın: hangi iki farklı ileti çıkıyor ve neden ikisi de gerekli?

H11_A03_hata_bul_sinir.c · satır 15–23

```c
    int m[SATIR][SUTUN] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    int toplam = 0;

    for (int i = 0; i < SATIR; i++) {
        for (int j = 0; j <= SUTUN; j++) {
            toplam += m[i][j];
        }
    }
    printf("Toplam: %d\n", toplam);
```

Alıştırma 11.4 ★★☆ *Hata bul*

`H11_A04_hata_bul_devrik.c` 3 × 3 matrisin (1 … 9) yerinde devriğini almalı, ama sonra yazdırdığı matris özgün matrisin aynısı. Neden? Tek karakterlik düzeltme yeter.

H11_A04_hata_bul_devrik.c · satır 15–21

```
15      for (int i = 0; i < N; i++) {
16          for (int j = 0; j < N; j++) {
17              int gecici = k[i][j];
18              k[i][j] = k[j][i];
19              k[j][i] = gecici;
20          }
21      }
```

C) Parsons problemi ve D) kod tamamlama

Alıştırma 11.5 ★★☆ *Parsons*

`sutun_toplamlari` fonksiyonunun satırları karışık. Doğru sıraya dizin ve girintileyin; **bir satır fazladır**. Beklenen çıktı: `15 18 21 24`.

H11_A05_parsons_sutun_toplam.c · satır 12–21

```
        sonuc[j] = 0;
            sonuc[j] += m[i][j];
void sutun_toplamlari(int m[][SUTUN], int sonuc[])
    for (int j = 0; j < SUTUN; j++) {
        }
        for (int i = 0; i < SATIR; i++) {
            sonuc[i] += m[i][j];
    }
{
}
```

Alıştırma 11.6 ★★☆ *Kod tamamla*

(2 × 3) · (3 × 2) çarpımındaki dört boşluğu (`____`) doldurun. Beklenen çıktı: `58 64` ve `139 154`.

H11_A06_kod_tamamla_carpim.c · satır 14–21

```
14      for (int i = 0; i < 2; i++) {
15          for (int j = 0; j < ____; j++) {
16              c[i][j] = ____;
17              for (int k = 0; k < ____; k++) {
18                  c[i][j] += a[i][k] * b[____][j];
19              }
20          }
21      }
```

E) Kod yazma

Alıştırma 11.7 ★★☆ *Kod yaz*

Pascal üçgeni. `int p[7][7]` dizisinde Pascal üçgeninin ilk 7 satırını hesaplayıp yazdırın: `p[i][0] = p[i][i] = 1`, içeride `p[i][j] = p[i-1][j-1] + p[i-1][j]`. Yalnızca alt üçgeni (j ≤ i) kullanın. Son satır `1 6 15 20 15 6 1` olmalı.

Alıştırma 11.8 ★★☆ *Kod yaz*

Sihirli kare. `int sihirli_toplam(int m[][N])` yazın: bütün satır, sütun ve iki köşegen toplamı aynıysa bu toplamı, değilse -1 döndürsün. Dürer'in Melencolia I gravüründeki kareyle (16 3 2 13 / 5 10 11 8 / 9 6 7 12 / 4 15 14 1) deneyin; iki sayının yerini değiştirince -1 dönmeli.

Alıştırma 11.9 ★★★ *Kod yaz*

90° döndürme. N × N matrisi saat yönünde 90° döndürün: (a) yeni bir matrise `d[j][N-1-i] = m[i][j]`; (b) yerinde: önce devrik (Örnek 4), sonra her satırı ters çevirin (H10). İki yöntemin aynı sonucu verdiğini gösterin.

Alıştırma 11.10 ★★★ *Kod yaz*

Taşkın doldurma (flood fill). `char g[6][10]` haritasında `'#'` duvar, `'.'` boş hücredir. Verilen bir hücreden başlayıp **4-komşuluk** ile bağlı bütün boş hücreleri `'~'` yapan ve doldurulan hücre sayısını döndüren **özyinelemeli** (H9) bir fonksiyon yazın. Temel durumlar: ızgara dışı, duvar, zaten dolu. Duvarla ayrılmış bölge değişmemeli.

Alıştırma 11.11 ★★☆ *Kod yaz*

Atın hamleleri. 8 × 8 satranç tahtasının her karesi için, oradaki bir atın tahtadan çıkmadan gidebileceği kare sayısını hesaplayın (8 L hamlesi iki uzaklık dizisinde, H11_K07 gibi). Köşe 2, merkez 8 olmalı; bütün tablonun toplamını da yazdırın.

Alıştırma 11.12 ★★★ *Kod yaz*

Simit (torus) Hayat Oyunu. Örnek 13'ü, ızgaranın kenarları birbirine bağlıymış gibi değiştirin: komşu indisi `(i + di + SATIR) % SATIR` ile sarılır, sınır denetimi gerekmez. 6 × 6 ızgarada planörün kaçıncı nesilde başladığı yere döndüğünü programla bulun.

9. Kendini Değerlendirme

- ☐ İki boyutlu diziyi bildirebiliyor, satır satır ve kısmi ilk değer verebiliyor, `sizeof` ile satır ve sütun sayısını bulabiliyorum.
- ☐ İç içe döngüyle bir matrisi satır satır ve sütun sütun dolaşabiliyor, satır ve sütun toplamı/ortalaması hesaplayabiliyorum.
- ☐ Satır öncelikli yerleşimi açıklayabiliyor ve `&m[i][j]`'nin uzaklığını (i × SUTUN + j) × eleman boyutu ile hesaplayabiliyorum.
- ☐ İki boyutlu diziyi fonksiyona aktarırken sütun boyutunun neden zorunlu olduğunu açıklayabiliyorum; bir satırı 1B fonksiyona verebiliyor, bir sütunu kopyalayabiliyorum.
- ☐ Matris toplama, devrik, çarpma, köşegen, simetri ve üçgen denetimlerini yazabiliyorum.
- ☐ Izgara problemlerinde 4 ve 8 komşuluğu sınır denetimiyle uygulayabiliyor; filtre ve Hayat Oyunu'nda neden ayrı bir çıktı matrisi gerektiğini açıklayabiliyorum.
- ☐ Satır ve sütun öncelikli dolaşmanın süre farkını önbellek satırıyla açıklayabiliyor, `clock()` ile güvenilir bir ölçüm tasarlayabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · Bu haftanın yapı taşları AP-2'de nerede?

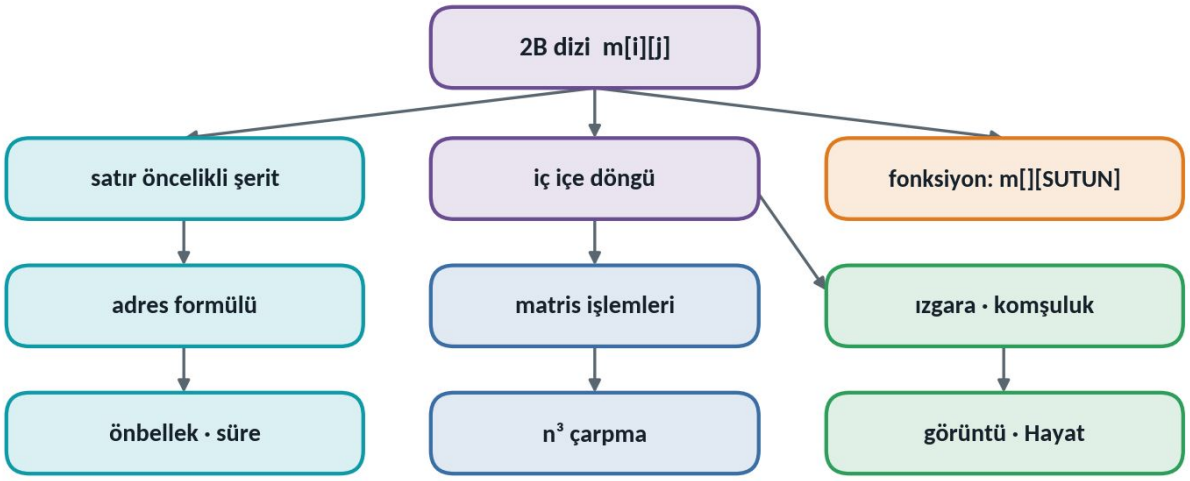
- Matris çarpımının n^3 işlem sayısı, AP-2'de polinom karmaşıklık sınıflarının somut örneğidir.
- Izgara komşuluğu, ileride graf dolaşma algoritmalarına (taşkın doldurma, BFS/DFS) açılan kapıdır.
- 2B tablolar, AP-2'de dinamik programlama tablolarının doğal temsilidir.

Bu derste yalnızca işlem **sayıyor** ve **ölçüyoruz**; biçimsel analiz AP-2'dedir. **Merak:** C satır önceliklidir; Fortran, MATLAB ve Julia çok boyutlu dizileri **sütun öncelikli** (column-major) yerleştirir: orada iç döngünün satır indisinde dönmesi hızlıdır, Örnek 12'nin tersi.

11. Özet, Kavram Haritası ve Terimler

Büyük Soru'nun yanıtı. Fotoğraf, satranç tahtası ya da sınıf listesi bilgisayar için **iki boyutlu bir dizidir**. Ama bellek tek boyutludur: C tabloyu **satır öncelikli** bir şerit olarak yerleştirir ve `m[i][j]`'yi taban + (i × SUTUN + j) × eleman boyutu adresinde bulur. Satır satır dolaşma şeridi ardışık okur ve önbelleğe getirilen her 64 baytı kullanır; sütun sütun dolaşma her adımda bir satır boyu atlar. 64 MB'lık matriste aynı toplam bu yüzden -O0'da 2,8, -O2'de 15,1 kat farklı sürede hesaplandı.

Şekil 11.8
Hafta 11 kavram haritası



Tablo 11.7
Terim sözlüğü (yol haritası Bölüm 6.8 ile tutarlı)

İngilizce	Türkçe karşılık	Bu haftada
two-dimensional array; matrix	iki boyutlu dizi; matris	`int m[SATIR][SUTUN]`
row-major order / column-major order	satır öncelikli / sütun öncelikli yerleşim	C satır önceliklidir
transpose; identity matrix; symmetric	devrik; birim matris; simetrik	Örnek 4–5
neighborhood; bounds checking	komşuluk; sınır denetimi	4 ve 8 komşu
pixel; grayscale; threshold; blur	piksel; gri ton; eşikleme; bulanıklaştırma	Örnek 11
cache; cache line; locality	önbellek; önbellek satırı; yerellik	64 bayt = 16 int

Sonraki hafta (H12, İşaretçiler). `m[1]` bir satırın **adresiydi** ve fonksiyona yalnızca adres gidiyordu. H12 adresleri bir değişkende tutmayı öğretecek: **işaretçi**. `a[i]` ile `*(a + i)` eşdeğerliği, `m[i]`'nin neden SUTUN × 4 bayt atlayarak ilerlediği, matris boyutunu çalışma zamanında seçmek (`malloc`) ve Mini-OBS'nin sabit kapasitesinin kalkması (v0.12) orada.

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

Durumeric, Z., Li, F., Kasten, J., Amann, J., Beekman, J., Payer, M., Weaver, N., Adrian, D., Paxson, V., Bailey, M., & Halderman, J. A. (2014). The matter of Heartbleed. In *Proceedings of the 2014 Internet Measurement Conference* (pp. 475–488). ACM. https://doi.org/10.1145/2663716.2663755

Gardner, M. (1970). Mathematical games: The fantastic combinations of John Conway's new solitaire game "life". *Scientific American, 223*(4), 120–123. https://doi.org/10.1038/scientificamerican1070-120

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design RISC-V edition: The hardware/software interface*. Morgan Kaufmann.

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

Uecker, M. (2020). *Compatibility of pointers to arrays with qualifiers* (WG14 N2607). ISO/IEC JTC1/SC22/WG14. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2607.pdf